Assurance Case Outline v1.2 Specification

3 April 2026

A user-friendly, intuitive, indentation-based assurance case outline format designed for authors, supporting automatic transformation into an internal Node Model, APL (Assurance Pattern Language) blocks, and graphical representations. A prototype implementation for Assurance Case Outline (ACO) is part of the Open Evidential Tool Bus (O-ETB) developed for the EC MedSecurance Project.

This v1.2 revision is a conservative evolution of v1.1. It preserves the look and feel of the ACO language previously defined, with modest refinements, while clarifying and extending the behaviour of the accompanying processor (canonicalisation, diagnostics, ASCII tree view), and formalising author guidance and assessment concepts that are already used operationally in the O-ETB workflow. Material newly introduced or substantially revised for v1.2 is highlighted in blue.

Contents

1 Purpose and Scope **1**

2 Core Concepts **2**
2.1 Node Types . . . 2
2.2 Handling of Provisional Evidence . . . 3
2.3 Node Header Syntax . . . 5
2.4 Node Body Text . . . 6
2.5 Unit Headers vs. Node Headers . . . 7
2.6 Optional Case Unit Header . . . 8
2.7 Case Metadata, Modules, and Anonymous Cases . . . 9
2.8 Module Unit Header . . . 10
2.9 ModuleRef and GoalRef Nodes: Syntax and Semantics . . . 10

3 Indentation-Based Structure **12**
3.1 Indentation Rules . . . 12
3.2 Default Relationship Semantics . . . 12

4 Optional Natural-Language Relations **13**
4.1 Supported Relations . . . 13
4.2 Semantics and Interaction with Indentation . . . 13

5 Node States **14**

6 Evidence Semantics and Provisional Categories **14**

7 Style and Clarity Guidelines **15**
7.1 Bodies of Goal and Module node headers . . . 15
7.2 Placeholders . . . 15
7.3 Naming Rules . . . 15
7.4 Text Blocks . . . 16

8 Errors and Warnings **16**
8.1 Errors (Syntax / Semantics / Structural) . . . 16
8.2 Warnings (Ambiguity, undefined placeholders, undeveloped nodes) . . . 17

9 Worked Example — Operational Plane Pattern **18**
9.1 Outline Example . . . 18
9.2 Derived Node Model (Semantic Graph) . . . 18

10 Mapping to Argument Pattern Language (APL) **19**
10.1 General Mapping Principles . . . 19

10.2 Example Translation to APL . . . 20

11 Visualisation and Rendering **20**

12 ACO Processor and Tooling Context **21**

12.1 Parsing and Node Model Construction . . . 21

12.2 Canonicalisation and Baseline ACO Files . . . 22

12.3 Diagnostics and Statistics . . . 22

13 Iterator-Bearing Strategies and Validation Boundaries **27**

13.1 Iterator-bearing Strategy syntax . . . 27

13.2 Validation boundaries . . . 27

13.3 Term-valued iterands . . . 28

13.4 Validation API and provider architecture direction . . . 28

14 Module declarations versus module references **28**

15 Summary **28**

16 Looking Forward **29**

16.1 Future Extensions . . . 29

16.2 Assessment Purpose . . . 30

16.3 Assessment and Incremental Improvement . . . 30

16.3.1 Assessment Dimensions (Vector-Valued) . . . 30

16.3.2 Confidence as a Derived Measure . . . 31

16.3.3 Evolutionary / Search-Based Improvement Model . . . 31

16.3.4 Author Guidance Modes . . . 31

16.4 Visualisation Support and Levels . . . 32

16.5 Where State Lives in the Toolchain . . . 32

16.6 Role of AI Assistance (Out of Scope for current efforts) . . . 33

17 Informative Note on the Emerging ACO Workbench Action Layer **34**

17.1 Actions and candidates . . . 34

17.2 Constructor basis from an initially empty outline . . . 34

17.3 Targets and pairwise loci . . . 34

17.4 Current transformation family and diagnostics . . . 35

17.5 Expressive completeness and structural abstractions . . . 35

1 Purpose and Scope

This specification defines a structured textual "Assurance Case Outline" (ACO) language that is domain-independent and platform-neutral for authoring assurance case instances and modules. It extends toward the user the interface presented by the O-ETB, providing a gentle entry point to its use that does not involve the technical syntax of its Argument Pattern Language (APL).

ACO is intended for use by assurance engineers and assurance case authors in conjunction with O-ETB, though it is not tied to O-ETB internals, but is compatible by translation with O-ETB's Argument Pattern Language (APL) concepts and with the GSN Community Standard where that does not conflict with the current O-ETB implementation.

The ACO notation aims to balance:

- Ease of authoring (minimal punctuation, close to "normal" outlines).
- Readability (GSN-like, hierarchical structure).
- Machine-interpretability (convertible to APL, DOT/Graphviz, and other formats).

The **ACO outline is a human-friendly avenue of access to assurance case automation provided by O-ETB**. APL and various visualisations are generated from it by the ACO Processor.

In particular:

- **ACO is unambiguous enough** to be parsed into a precise internal representation (Node Model).
- The Node Model is a precursor to, and can then be systematically mapped into, APL blocks and graphical representations.
- The Node Model is also the place where checks, diagnostics, and transformations are performed.

Throughout this specification, reference is made to a front-end tool that parses, canonicalises, and analyses ACO outlines. We currently refer to this tool as the *ACO Frontend*, and, where the context is clear, simply *Frontend*. The Frontend is a *reference implementation* of the present specification, intended to demonstrate the intended interpretation of the ACO language and to support authoring, diagnostics, and integration with downstream tooling.

Envisioned extensions to the current ACO processor will give it further capabilities, distinct from those provided by the O-ETB "backend," in support of the initial framing and iterative and incremental construction and refinement of assurance cases. Once those extensions have been implemented we plan to refer to them, along with the current ACO processor functions, as the "ACO Workbench" (or just "Workbench").

The reference implementation of the Frontend:

- parses ACO into the Node Model, assigning to each node a unique identifier (ID),
- computes hierarchical IDs that reflect the indentation structure,
- can canonicalise an ACO outline (systematically and consistently renumbering IDs), and
- can quickly generate several "ASCII tree views" for easier inspection, comprehension and review.

Version 1.2 treats input assessment and author guidance as first-class concerns: the front-end is expected to compute and report a set of preliminary quality signals (e.g. undeveloped nodes, provisional evidence, uninstantiated nodes) that can be used both for expert review and for future "wizard" guidance for inexperienced authors as part of the Workbench.

---

2 Core Concepts

An assurance case outline is constructed of one or more Nodes. Each Node is represented as one or more text lines. Nodes are grouped into units that are either a Case or a Module.

2.1 Node Types

ACO adopts a minimal, GSN-inspired set of node types, that is also aligned with APL concepts:

- **Goal** — a claim that must hold.
- **Strategy** — a structural combinator explaining how child nodes collectively support a parent goal node (may be optional in early usage).
- **Context** — relevant facts, definitions, scope, and environmental conditions.
- **Assumption** — asserted conditions that are taken as given about another node.
- **Justification** — rationale explaining why an approach is acceptable.
- **Evidence** — an artefact or reference that directly discharges the goal it supports.
- **Module** — a reference to a unit that encapsulates a goal and the structure that supports it, serving as:
  - a reusable argument fragment (possibly parameterised),
  - a component-level argument, or
  - an evidence-aggregation block.

In v1.2 revision 07, a further distinction is made between a top-level `Module:` unit header, which introduces a reusable module definition, and an in-tree `ModuleRef` occurrence, which invokes such a module within a local argument subtree. The specialised leaf form `GoalRef` is also recognised for local shared-goal reuse. These additions do not displace the core GSN-inspired node family, but they refine how module invocation and local structural aliasing are expressed in author-facing ACO.

Evidence, ModuleRef, and GoalRef Nodes are *leaf-level* elements of an assurance case, though different kinds of leaves. In particular:

- an Evidence, ModuleRef, or GoalRef occurrence may appear as a child of a Goal or Strategy Node,
- an Evidence or ModuleRef occurrence may contain body text describing the artefact, its provenance, or how it supports the associated claim; a GoalRef occurrence is a compact alias form and normally has no body text, but

- no Goal, Strategy, Context, Assumption, Justification, Evidence, ModuleRef, or GoalRef node may appear beneath an Evidence, ModuleRef, or GoalRef occurrence in the outline,
- Context, Assumption or Justifications related to evidence or module references should instead be attached to the parent of the Evidence, ModuleRef, or GoalRef occurrence.

An Evidence Node does not introduce further decomposition structure beneath itself in the ACO indentation tree and therefore **must not have child nodes** in the indentation structure (though it may have Body lines at one greater level of indentation).

In accordance with established GSN practice, a ModuleRef occurrence in ACO is a leaf node with respect to the enclosing argument. Decomposition and supporting structure occur within the referenced module unit, not under the module reference itself. Modular arguments allow a complex argument to be decomposed into independent, reusable argument fragments.

A GoalRef occurrence is also a leaf-level form. It does not introduce a new supporting subtree; rather, it denotes local reuse of an already established goal subtree within the same unit.

An Evidence Node says, "justification ends here," while a ModuleRef Node says, "justification continues (or is completed) elsewhere," and a GoalRef Node says, "the same supporting argument already established elsewhere in this unit is being intentionally reused here." Tools should treat any attempt to introduce children under an Evidence, ModuleRef, or GoalRef occurrence as a structural error, or at least emit a strong warning in permissive modes. This restriction reflects the intended role of Evidence as a direct claim discharger rather than a decomposition construct.

Additional specialised constructs (e.g. richer module structures, structural abstractions or domain-specific node flavours) can be introduced later, but the initial ACO v1.2 family focuses on these core types.

(v1.2 clarification) Evidence nodes are intended to be leaf-level dischargers of a goal: they do not introduce further decomposition structure beneath themselves in the ACO indentation tree. Tools may warn (or treat as an error in strict modes) if an Evidence, ModuleRef, or GoalRef occurrence has children.

2.2 Handling of Provisional Evidence

During the development of an assurance case it is common for evidence to exist in varying states of maturity. ACO explicitly supports this reality by allowing *provisional evidence* to appear in an outline.

Provisional evidence does not yet have a precisely defined validation method or a procedure for generating the artefact that is to serve as the evidence datum. Therefore, provisional evidence is not yet fully accepted as discharging the claim it supports, but is nevertheless recorded to guide development, planning, and review. Evidence nodes with evidence categories having provisional status are not considered to be fully developed.

Representation in ACO. Provisional status is not expressed through a distinct node type or specialised syntax. Instead:

- provisional evidence is represented using ordinary `Evidence` nodes,
- the provisional status of an evidence category is determined by the absence of its inclusion in a set of recognized evidence categories for which the validation method is well-defined,
- the permissibility of the appearance of provisional evidence categories is determined by a configuration option of the tooling,

- its provisional nature may be further conveyed through the short label, body text, annotations, or tool-specific conventions, and
- the core ACO syntax and indentation rules apply unchanged.

This design keeps the language syntax stable while allowing tools and workflows to interpret evidence maturity.

Typical Provisional Categories. Tools may recognise a number of provisional evidence categories, such as:

- **Planned** – evidence expected to be produced in the future,
- **Placeholder** – a temporary stand-in used to reserve a position in the argument,
- **Candidate** – evidence under evaluation or review, and
- **Accepted** – evidence judged sufficient to support the associated claim.

These categories are indicative rather than normative; the ACO specification does not mandate specific labels or keywords for these characteristics.

Semantic Interpretation. From the perspective of the ACO language:

- evidence having provisional and accepted evidence categories are structurally equivalent,
- both kinds of evidence categories may appear as children of Goals, Strategies, or Modules, and
- both contribute `supported_by` relations in the Node Model.

However, tools should distinguish provisional evidence from accepted evidence when assessing the *maturity* or *completeness* of an assurance case. Also, when an ACO containing a provisional evidence node is translated into APL, in the instantiation of an assurance case from the resulting APL the resulting evidence block will carry the provisional status of the evidence and a validation procedure will not be invoked.

Tool Responsibilities. When a provisional evidence category is detected, tools should:

- classify and report provisional evidence explicitly,
- avoid treating provisional evidence as fully discharging a claim in maturity assessments, and
- support workflows in which provisional evidence is progressively replaced or upgraded to an item belonging to a fully defined evidence category as the case evolves.

The presence of provisional evidence should not be treated as a structural error. It is an expected and useful feature during iterative assurance case development.

2.3 Node Header Syntax

Each node is introduced by a **header line**, followed by one or more indented **body lines**. The header has the general form:

```
<Type> [ID] <shortLabel>:
```

where:

- `<Type>` is one of:
  - `Goal`
  - `Strategy`
  - `Context`
  - `Assumption`
  - `Justification`
  - `Evidence`
  - `Module`
- `[ID]` is an **optional** identifier, such as `G0`, `G1`, `C3`, `M2`, etc.
  - If present, it is a single token (no spaces).
  - If absent, the header consists only of `<Type> <shortLabel>:`.
- `<shortLabel>` is a concise human-oriented label that identifies the node. It may be written in one of two forms:
  - **Unquoted short label**: a single token with an initial alphabetic character and no spaces.
    - CamelCase or snake_case is recommended (e.g. `operationalPlane`, `componentBehaviour`, `plane_definition`).
  - **Quoted short text**: a short quoted string enclosed in double quotes, allowing spaces.
    - This form is intended for cases where a compact but readable phrase is preferable to an identifier-style token.
- The header line **must end** with a colon (`:`). (v1.2 extension) This restriction has been relaxed to enable a compact and natural beginning of the Body text. Text following the colon is treated as the first line of the Body and effectively rewritten as such.

Two specialised reference forms are additionally recognised in revision 07:

- `ModuleRef [ID] [<callExpr>]:` — an in-tree module invocation. The bracketed call expression denotes the referenced module or pattern together with any actual arguments. A short explanatory body may follow in the ordinary way after the colon.
- `GoalRef <TargetId>` — a compact local shared-goal reference. This is a leaf form, normally written without a trailing colon or body text. It denotes reuse of a goal subtree already established elsewhere in the same unit.

Thus, the following header forms are all valid:

With ID and unquoted label:

```
Goal G0 operationalPlane:
```

With ID and quoted short text:

```
Goal G0 "Operational Plane Policy":
```

Without ID and unquoted label:

```
Context planeDefinition:
```

Without ID and quoted short text:

```
Context "Plane Definition":
```

Any of these with Body text following the colon, e.g.:

```
Goal G0 operationalPlane: The operational plan satisfies its {specification}.
```

Parsing conventions (informal):

- The first token is always the **Type**.
- If the second token matches the pattern of an ID (e.g. `G<digit>+`, `C<digit>+`, `M<digit>+`, etc.), it is treated as the **ID**.
- The **shortLabel** is then:
  - the next single token, if unquoted, or
  - the complete quoted string, if the next non-whitespace character is a double quote.
- Everything between the end of the shortLabel and the trailing colon `:` must be whitespace only.

Tools should preserve the shortLabel text as written (including spaces in the quoted form) when generating diagnostics, ASCII tree views, and downstream artefacts. Canonical identifiers derived from indentation are independent of the surface shortLabel representation.

2.4 Node Body Text

The header line is followed by one or more **indented body lines**, containing a natural-language description. As noted in the Node Header section above, there is an exception for the first (or only) line of the body to begin after the colon on the node header line.

Examples:

```
Goal G0 operationalPlane:
  The operational plane guarantees the {local policy} is met.

Context C0 planeDefinition:
  The system model describes the plane and its {properties}.

Module M1 component:
  Behaviour of the {component} meets its local policy.
```

Rules:

- Body lines must be indented **more** than the header (one indentation level deeper).
- Body text may span multiple lines.
- Blank lines are allowed between nodes for readability, but should not break the structural semantics.
- The first (or only) line of body text may appear following the colon on the associated node header line.

2.5 Unit Headers vs. Node Headers

ACO distinguishes two kinds of "headers":

- **Unit headers** begin a new *unit*:
  - `Case: ...`
  - `Module: ...` (a module *unit* header)

  Unit headers are always written at indentation level 0 and they reset the unit-local scope of node identifiers (IDs).
- **Node headers** declare nodes within the current unit (e.g. `Goal`, `Strategy`, `Context`, `Assumption`, `Justification`, `Evidence`, and `Module <ID> ...:`). Node headers may appear at any indentation level and do not start a new unit.

In revision 07, an in-tree reference to a module is introduced using the node header syntax with the type `ModuleRef`. The earlier local occurrence form using `Module` is still accepted for backward compatibility but is retained only as a compatibility alias. New outlines should use `ModuleRef`.

The general form of a module reference header is therefore:

```
ModuleRef [ID] [<callExpr>]:
```

where `<callExpr>` is typically a module or pattern name with actual arguments, for example `[n_foundational_plane_node({S},{N})]`. The previous compatibility form `Module [ID] [<callExpr>]:` remains acceptable to the current processor but is no longer the preferred author-facing syntax.

where:

- `[ID]` is an optional identifier, subject to the same rules as for other node types.
- `<shortLabel>` is the *module name*, written either as an unquoted single token or as a quoted short text, as described in Section 2.2.

Example:

```
ModuleRef M1 [componentBehaviour({component})]:
  Behaviour of the {component} meets its local policy.
```

or, using a quoted module name:

```
ModuleRef M1 [componentBehaviour({component})]:
  Behaviour of the {component} meets its local policy.
```

A ModuleRef occurrence participates in the indentation-defined structure in exactly the same way as other node types. By default:

- a ModuleRef occurrence appearing as a child of a Goal contributes a `supported_by` relation, and
- the referenced Module unit may itself have supporting children (e.g. sub-goals, strategies, or evidence), subject to the general indentation rules.

Modules are *optional* in an ACO outline: an assurance case may be written entirely without modules. However, whenever a module is introduced, it must be declared explicitly using a `Module:` unit header or a local `ModuleRef` occurrence as appropriate; there is no implicit or anonymous module construct.

Although modules are often used to represent reusable argument fragments or patterns, ACO v1.2 does not distinguish syntactically between "parameterized modules" and other kinds of concrete modules that may be used solely to break up an assurance case into more manageable pieces for description or presentation. Any additional semantics related to reuse, instantiation, or parameterisation are handled by tooling and by downstream representations (such as APL), not by the surface ACO syntax itself.

2.6 Optional Case Unit Header

In larger and more developed outlines it is often useful to name the overall assurance case explicitly. ACO therefore supports an optional *case header* at the top of a case unit, written in a simple free-text form:

```
Case:  <Title> - <Scope>
```

where:

- `<Title>` is a short human-readable name for the assurance case, and
- `<Scope>` (optional) briefly summarises the scope, system boundary, or context of the argument.

Variants such as `CASE: ...` are accepted. The separator between title and scope is a dash surrounded by spaces; if no dash is present, the entire text after `Case:` is treated as the title and the scope is treated as empty.

The case header is *metadata*, not a node in the argument graph:

- it does not participate in `supported_by` or `in_context_of` relations,
- it does not affect indentation-based structure, and
- it is not assigned a node identifier.

Absence of a Case Header. If no `Case:` header is present, the ACO file is treated as describing an *anonymous assurance case*. In this situation:

- the outline is still fully valid ACO,
- all nodes and relations are interpreted normally, and
- the top-level nodes collectively constitute the case without an explicit name.

Tools should handle anonymous cases gracefully. In particular:

- generated artefacts (e.g. reports, DOT titles, APL block sets) may use a default or tool-generated case name,
- diagnostics and statistics should not require the presence of a case header, and
- the absence of a case header should not be reported as an error or warning.

The case header exists solely to improve clarity, traceability, and presentation. Its use is recommended for non-trivial assurance cases, but it is not mandatory.

2.7 Case Metadata, Modules, and Anonymous Cases

An ACO file may include a `Case:` header, one or more `Module` nodes, both, or neither. The appearances of these constructs are not mutually exclusive and serve different purposes.

Case Header and Modules. The `Case:` header, when present, provides a name and optional scope for the overall assurance case. Module headers, when present, provide structure within the case by encapsulating reusable or composite argument fragments.

It is therefore valid for an ACO file to:

- declare a case using a `Case:` header and contain one or more Module headers,
- declare a case and contain no modules,
- contain modules without declaring an explicit case header, or
- contain neither a case header nor any module headers.

Anonymous Cases. If no `Case:` header is present, the outline is interpreted as describing an *anonymous assurance case*. In this situation:

- the top-level nodes collectively define the case,
- all structural and semantic rules apply unchanged, and
- the absence of a case name has no effect on correctness or completeness.

Tool Interpretation. Tools should treat the case header strictly as metadata:

- it does not participate in the Node Model,
- it does not constrain the presence or absence of Module units or ModuleRef occurrences, and
- it should not be required for parsing, canonicalisation, or analysis.

When a Case: header is absent, tools may generate a default or derived case identifier and empty scope for presentation purposes (e.g. in reports or visualisations), but this is not treated as semantically significant.

This separation allows ACO to support both named assurance cases and reusable, case-agnostic module libraries using a single uniform syntax.

2.8 Module Unit Header

ACO supports the explicit declaration of reusable or composite argument structures through *Module* unit headers. The distinction between a module node header and a module unit header is to be stressed. The former is a *reference* to a module while the latter introduces the *definition* of a module.

A Module encapsulates a goal together with the structure that supports it, and may represent, for example, a reusable argument fragment, a component-level argument, or an evidence-aggregation block. The occurrence of a Module header is not a module node, but rather metadata similar to a Case header.

2.9 ModuleRef and GoalRef Nodes: Syntax and Semantics

ModuleRef nodes provide a way to invoke structured argument fragments within an ACO outline. They are particularly useful for reuse, decomposition by component or subsystem, and for managing large arguments. GoalRef provides the corresponding compact in-unit notation for reusing a goal subtree that has already been established locally.

Syntax Recap. A ModuleRef node is introduced using the reference header syntax:

```
ModuleRef [ID] [<callExpr>]:
```

The syntactic rules for `[ID]` are the same as for other node types. The `<callExpr>` field is a term-like call designator rather than a free short label and is intended to lower deterministically into the corresponding APL pattern-reference form.

Structural Semantics. Semantically, a ModuleRef occurrence behaves as a specialised form of Goal support:

- a ModuleRef occurrence denotes a referenced module whose root claim is intended to be supported internally,
- its supporting structure is provided within the referenced module unit rather than as local indentation children, and
- when a ModuleRef occurrence appears as a child of another node, it contributes a `supported_by` relation.

Thus, from the perspective of the Node Model, a ModuleRef occurrence participates in the same local support relations as a Goal-like child while remaining a leaf in the enclosing unit.

Internal Structure. A ModuleRef occurrence must not have local structural children. The referenced Module unit may of course contain children of any node type permitted by the general rules, including:

- Goals or sub-goals,
- Strategies explaining internal decomposition,
- Contexts, Assumptions, and Justifications, and
- Evidence nodes (subject to the restriction that Evidence nodes are leaves).

This allows a Module unit to encapsulate a complete argument fragment.

Use and Interpretation. ModuleRef and GoalRef are optional: an ACO outline need not use them at all. When they are used, their intended interpretation is:

- to make the argument structure more explicit and manageable,
- to support reuse of argument fragments across cases or components, and
- to provide a natural boundary for later instantiation, parameterisation, or mapping to downstream representations.

GoalRef has an additional local semantic constraint: the referenced goal identifier must denote a goal already established within the same unit. The current processor resolves GoalRef using that unit-local symbol table discipline. A future semantic check will further confirm that the assumptions and contexts required by the reused subtree are satisfied at each reuse site.

ACO v1.2 does not require the author to write internal ETB terms such as `goal_ref/1` or `ac_pattern_ref/5`. Those are downstream forms handled by translation to APL and instantiation in O-ETB.

Tool Considerations. Tools should:

- treat ModuleRef occurrences as first-class reference nodes in the argument graph,
- include them in diagnostics and statistics,
- allow ModuleRef occurrences to appear wherever a Goal could appear structurally, and
- avoid imposing additional constraints not stated in this specification.

This deliberately lightweight treatment ensures that ModuleRef and GoalRef can be used flexibly while remaining compatible with both informal authoring workflows and more formal downstream analyses.

---

3 Indentation-Based Structure

3.1 Indentation Rules

Indentation is the primary way to represent structure and relationships in an outline.

- Indentation uses **2 spaces per level** by default.
  - Tabs are discouraged; tools should normalise to spaces or flag tabs.
- A node header at indentation level k ($k \geq 0$):
  - If $k = 0$, it is a top-level node.
  - If $k > 0$, it is structurally a **child** of the nearest preceding node header at indentation level $k - 1$.
- When the indentation decreases from level k to level j:
  - All preceding nodes at levels $> j$ are considered "closed".
  - The new node at level j is attached (as child or sibling) appropriately in the tree implied by indentation level.
- Indentation must never "jump" to a greater level by more than one level without passing through the intermediate levels. For example:
  - Going directly from level 0 to level 3 is **invalid**.
  - This is treated as a structural error (see Section 8).

3.2 Default Relationship Semantics

From the indentation-defined tree, ACO derives **default relationships**.

For a parent node P and a child node N (implied by indentation):

- If N is of type Goal, Strategy, Evidence, Module, Assumption, or Justification:
  - A default edge `supported_by(P, N)` is added to the Node Model.
- If N is of type Context:
  - A default edge `in_context_of(P, N)` is added.

This means:

- Authors do **not** need to explicitly spell out relations for the common tree-structured argument.
- Non-tree or cross-cutting relations can be added explicitly using natural-language relation sentences (see Section 4).

4 Optional Natural-Language Relations

To improve readability and to express **non-hierarchical** (non-tree) relations, authors may add optional explicit relation sentences.

These are written in a natural language style but parsed into relations over node IDs.

4.1 Supported Relations

The following patterns are accepted (case-insensitive, minor variations allowed).

Supported-by:

```
G0 is supported by G1, G2, C0 and M0.
G1, G2, C0 and M0 support G0.
```

In-context-of:

```
G0 is in context of C0.
C0 provides context for G0.
```

Where:

- Left-hand side and right-hand side refer to existing node IDs (e.g. `G0`, `C0`, `M0`).
- The IDs must have been previously defined as node IDs in headers of the permissible types within the unit with which the explicit relation sentence is associated.

(v1.2 robustness) Implementations should accept comma-separated lists with a final conjunction ("and"), and should accept simple plural verb forms (e.g. "provide context for") as synonyms of the singular forms, as long as the intended relation type remains unambiguous.

4.2 Semantics and Interaction with Indentation

- These explicit relation sentences **augment** the default relations implied by indentation. They may not contradict the default relations.
- They may:
  - explicitly reiterate links already implied by the hierarchy (e.g., to unambiguously clarify relations that may be difficult to identify upon inspection of the text due to vertical separation within the text), or
  - introduce additional support or context edges (e.g. providing relations that cut across branches of the tree).
- The parser adds these relations into the Node Model as:
  - `supported_by(parent, child)`
  - `in_context_of(parent, context)`

Dangling references (IDs mentioned in explicit relation sentences that have no corresponding node header) are treated as **errors** (see Section 8).

5 Node States

ACO v1.2 distinguishes the following independent node states, which must be tracked separately:

- **Uninstantiated node**: a placeholder referencing a future concrete entity. Examples include:
  - an "away goal" that refers to an external module/case not yet imported,
  - a module reference whose parameter bindings are not yet provided,
  - an evidence node that names an evidence category but lacks a concrete evidence instance.
- **Undeveloped node**: a concrete node whose argumentation is incomplete. Examples include:
  - a goal with insufficient subgoals/evidence,
  - a module that is present but has no internal supporting structure,
  - an evidence node whose category is permitted but has no validation semantics.

A node may be uninstantiated only, undeveloped only, or both. Tools must report these categories distinctly rather than collapsing them into a single "missing" or "incomplete" status.

---

6 Evidence Semantics and Provisional Categories

Within O-ETB, evidence is not merely a label: it has an intended semantic chain:

evidence category → validation method → agent/tool → status record

An evidence node is *fully developed* only when this chain is concretely established in the instantiated artefact repository.

ACO v1.2 introduces the notion of a *provisional evidence category*: a category atom that is not yet defined in the evidence-category knowledge base but is permitted (non-fatal) under a reserved provisional entry. Operationally:

- If the evidence category named by an evidence node is not defined in `KB/EVIDENCE/categories.pl`, but the knowledge base contains a reserved provisional entry (e.g. `evidence_category(pr_Type,_ValidationMethod)`), then the category is treated as *provisional*.
- Evidence with a provisional category is conceptually plausible but has no validation semantics yet; therefore it is treated as *undeveloped* (not merely "missing data") and must be surfaced to the author.

The reference front-end should count and report provisional evidence occurrences distinctly from other undeveloped evidence situations.

---

7 Style and Clarity Guidelines

7.1 Bodies of Goal and Module node headers

To keep the ACO both human-understandable and machine-interpretable:

- Goal and Module bodies should state **claims**, not vague aspirations.
- Avoid ambiguous terms such as:
  - "may", "might", "possibly", "etc.", "various", "as appropriate".
- Prefer measurable, testable statements, or at least well-scoped qualitative claims.

7.2 Placeholders

Placeholders are written in **braces**:

- `{component}`, `{local policy}`, `{properties}`, `{environment}`, etc.

Rules:

- Placeholders should be:
  - introduced within the unit (e.g. as a formal parameter), or
  - conventional/common in the domain, or
  - introduced and explained in a suitable Context node.
- Tools should emit **warnings** when:
  - placeholders appear that are never defined or are obviously inconsistent.

7.3 Naming Rules

- IDs (when used) must be **unique** within a single unit.
  - Recommended: prefix by type initial (e.g. `G0`, `G1`, `C0`, `M2`, etc.).
- `shortLabel` should:
  - contain no spaces,
  - start with an alphabetic character
  - preferably use camelCase or snake_case.
- The combination of Type, optional ID, and label should make each node uniquely identifiable.

Although authors are free to choose IDs, it is often convenient to follow a dotted or underscored, outline-style convention that mirrors the indentation tree, e.g. `G1.`, `G1.1`, `G1.1.2`, `G1`, `G1_1`, `G1_1_2`. The reference ACO front-end can compute such hierarchical IDs automatically from the indentation structure and compare them with author-supplied IDs, issuing diagnostic messages if an explicit dotted or underscored ID does not match the structure implied by indentation.

7.4 Text Blocks

- All node body text must be indented beneath the header (one indent level greater), for example:

```
Goal G1 componentBehaviour:
  This is body text for G1.
  It may span multiple lines.

Module M1 component:
  Behaviour of the {component} meets its local policy.
```

- Blank lines may be inserted for clarity, as long as they do not break the logical grouping of body text with its header.

---

8 Errors and Warnings

The ACO processor parses .aco input files into the Node Model and performs consistency checks. As a result of these checks the processor can issue error and warning messages.

8.1 Errors (Syntax / Semantics / Structural)

The following conditions are treated as **errors**:

- **Duplicate and undefined node IDs**:
  - The same ID used in more than one header line.
  - An ID that has not been defined in the unit is used in a relation sentence within the unit.
- **Dangling references**:
  - IDs used in explicit relation sentences that do not correspond to the ID of any well-formed node.
- **Indentation jumps**:
  - For example, going from level 0 directly to level 3 with no node at level 1 or 2.
- **Unknown node types**:
  - A header whose first token is not a recognised Type (Goal, Strategy, Context, Assumption, Justification, Evidence, or Module).
- **Malformed headers**:
  - Header lines missing the trailing (or separating) colon (`:`),
  - or with an ambiguous token sequence that cannot be parsed according to the rules in Section 2.2.

Such errors should prevent further processing until corrected.

8.2 Warnings (Ambiguity, undefined placeholders, undeveloped nodes)

The following conditions may trigger **warnings**:

- Ambiguous wording in Goal/Module bodies (use of "may", "etc.", etc.).
- Undefined placeholders in braces `{...}`.
- Goals that have neither supporting children (by indentation) **nor** explicit supporting evidence or modules.

Goals with no supporting structure are considered **Undeveloped**. To handle of undeveloped goals:

- Tools should:
  - treat these as a distinct category ("Undeveloped goals/modules"), and
  - optionally also report them as warnings.
- A report may, for example:
  - print a dedicated list of **Undeveloped** for fast review, and
  - also include them in the warnings list.

The ACO processor also computes simple statistics such as:

- total number of nodes and numbers by type (goals, strategies, contexts, assumptions, justifications, evidences, modules),
- numbers of `supported_by` and `in_context_of` edges arising from indentation vs. explicit relation sentences,
- counts of "cross-branch" relations where an explicit relation connects nodes that are not in an ancestor/descendant relationship in the indentation tree.

Note: "Explicit relation sentences" refer only to author-written relation sentences, regardless of whether they restate edges already implied by indentation. "Cross-branch" relations count only explicit relation edges that connect nodes that are not reachable via the indentation-derived tree edges (`supported_by` and `in_context_of`). Module references are treated as graph expansion and do not contribute to cross-branch relation counts.

These statistics are intended as feedback for authors and reviewers, not as part of the ACO language itself.

In addition, the front-end can emit *hierarchical ID diagnostics* for each node, indicating the canonical hierarchical ID derived from indentation and flagging any author-supplied dotted IDs that do not agree with that structure. Such diagnostics are warnings unless the user chooses to treat them as stricter constraints in a particular workflow.

(v1.2 extension) Tools should also distinguish and report *uninstantiated* nodes (placeholders referencing future concrete entities) separately from undeveloped nodes. Section 16.3 defines these states more precisely and explains how they factor into assurance case assessment and guidance.

---

9 Worked Example — Operational Plane Pattern

Below is a conformant example using only indentation-defined structure, plus some optional explicit relation sentences.

9.1 Outline Example

```
Goal G0 operationalPlane:
  The operational plane guarantees the {local policy} is met.

  Context C0 planeDefinition:
    The system model describes the plane and its {properties}.

  Goal G1 componentBehaviour:
    Compositional behaviour of separate components ensures the local policy is met.

    Module M1 component:
      Behaviour of the {component} meets its local policy.

  Goal G2 compositionBehaviour:
    Compositional behaviour of compositions ensures the local policy is met.

    Module M2 composition:
      Behaviour of the {composition} meets the local policy.

  Module M0 interface:
    Interaction between components and compositions is as defined by the security
    architecture interface.

G0 is supported by G1, G2, M0.
G0 is in context of C0.
G1 is supported by M1.
G2 is supported by M2.
```

9.2 Derived Node Model (Semantic Graph)

From this outline, the Node Model contains:

Nodes (with type, ID, and label):

```
(G0, goal,    operationalPlane)
(C0, context, planeDefinition)
(G1, goal,    componentBehaviour)
(G2, goal,    compositionBehaviour)
(M1, module,  component)
(M2, module,  composition)
(M0, module,  interface)
```

Relationships (edges):

```
supported_by(G0, G1)
supported_by(G0, G2)
```

```
supported_by(G0, M0)
in_context_of(G0, C0)
supported_by(G1, M1)
supported_by(G2, M2)
```

The Node Model is **the internal semantic representation** on which all further transformations (APL, diagrams, analysis) are based.

A canonicalisation tool, given this outline, can derive a hierarchical ID for each node from the indentation structure (e.g. `G1.`, `G1.1`, `M1.1.1`) and produce a canonical ACO file in which all node IDs follow that scheme and are consistent with their position in the tree. Authors may then adopt that canonicalised ACO as the new baseline, discarding earlier ad hoc IDs.

---

10 Mapping to Argument Pattern Language (APL)

The ACO notation itself is tooling-neutral, but it is designed to be systematically mapped to APL representation used by O-ETB.

10.1 General Mapping Principles

From the Node Model:

- Each node becomes an APL **block** with:
  - a block identifier (derived from the ID or a canonicalised label),
  - a block type (goal, strategy, context, assumption, justification, evidence, module),
  - a label and body text.
- Each `supported_by` edge becomes a corresponding **support** relation between blocks.
- Each `in_context_of` edge becomes a corresponding **context** relation between blocks.

The precise syntax of APL is defined by the O-ETB User Manual and implementation. The mapping must be:

- Deterministic: the same ACO outline always yields the same APL representation.
- Information-preserving: all node text and relations present in ACO must be recoverable from APL.

(v1.2 Note.) Earlier drafts referred to a distinct "pattern mode" of operation. This concept is no longer part of the ACO model. ACO v1.2 uses a single, uniform representation based on node types and indentation-defined structure, without a separate pattern mode. What are known as patterns in APL are represented as Module definitions in ACO.

Module definitions and other structured argument fragments may be mapped by tools into APL pattern definitions or instantiated APL cases, depending on context. Such mappings are performed by the toolchain and are outside the scope of the ACO syntax defined in this specification.

10.2 Example Translation to APL

The ACO example in Section 9.1 becomes the APL:

```
block(g0, goal,    operationalPlane,
      "The operational plane guarantees the {local policy} is met.").
block(c0, context, planeDefinition,
      "The system model describes the plane and its {properties}.").
block(g1, goal,    componentBehaviour,
      "Compositional behaviour of separate components ensures the local
       policy is met.").
block(g2, goal,    compositionBehaviour,
      "Compositional behaviour of compositions ensures the local policy
       is met.").
block(m1, module,  component,
      "Behaviour of the {component} meets its local policy.").
block(m2, module,  composition,
      "Behaviour of the {composition} meets the local policy.").
block(m0, module,  interface,
      "Interaction between components and compositions is as defined by
      the security architecture interface.").

supported_by(g0, g1).
supported_by(g0, g2).
supported_by(g0, m0).
in_context_of(g0, c0).
supported_by(g1, m1).
supported_by(g2, m2).
```

---

11 Visualisation and Rendering

This section discusses visual representations related to ACO and its role in the broader toolchain. It is important to distinguish between what is provided directly by the ACO Frontend and what is provided by downstream tools.

At present, the ACO Frontend does *not* generate a direct Graphviz DOT rendering of an ACO outline. ACO focuses on parsing, canonicalisation, diagnostics, and the production of intermediate representations suitable for further processing.

A DOT rendering of a complete assurance case is, however, provided by the O-ETB toolchain. In particular, the `export_case` command in O-ETB produces a DOT representation of the *final instantiated assurance case*, after all modules have been resolved, parameters instantiated, and case-level structure fixed.

Accordingly:

- ACO should be viewed as an authoring- and analysis-oriented front-end, not as a visualisation engine.
- Graphical views (such as DOT/Graphviz renderings) are generated from the instantiated case produced by O-ETB, not directly from raw ACO outlines.

This separation allows ACO to remain lightweight and focused, while enabling high-quality visualisations at the appropriate stage of the assurance workflow.

Example DOT Rendering For the example in Section 9.1, a DOT rendering might be:

```
digraph OperationalPlane {
  node [shape=box, style=rounded];

  G0 [label="Goal G0: operationalPlane"];
  C0 [label="Context C0: planeDefinition",
      shape=note, style="rounded,dashed"];
  G1 [label="Goal G1: componentBehaviour"];
  G2 [label="Goal G2: compositionBehaviour"];
  M1 [label="Module M1: component"];
  M2 [label="Module M2: composition"];
  M0 [label="Module M0: interface"];

  G0 -> G1;
  G0 -> G2;
  G0 -> M0;
  G0 -> C0 [style=dashed];
  G1 -> M1;
  G2 -> M2;
}
```

Styling (shapes, colours, fonts) is not part of the ACO spec, but visual conventions can help distinguish types (e.g. dashed edges for context).

(v1.2 note) When cross-branch relations are present (via explicit relation sentences), a DOT exporter may choose to draw them using a distinct style (e.g. dotted or coloured edges), or may emit a separate report listing cross-branch relations, to avoid obscuring the primary indentation-derived tree.

---

12 ACO Processor and Tooling Context

The ACO Processor (Frontend) is a reference implementation of this specification and is used in conjunction with the Open Evidential Tool Bus (O-ETB) in the MedSecurance Project.

The purpose of the reference implementation is to demonstrate the intended interpretation of the ACO language, to support authoring and analysis workflows, and to provide a concrete basis for integration with downstream tooling. It is not intended to be the only possible implementation, nor to normalise constraints beyond those defined by this specification.

12.1 Parsing and Node Model Construction

The ACO Processor:

- reads an ACO file (containing case and module units),
- strips comments and normalises indentation,

- identifies unit headers, node headers, bodies, and natural-language relation sentences,
- constructs a flat list of nodes of the form `node(Id, Type, Label, Body, Level, Line)`, and
- builds the tree edges implied by indentation, plus any additional edges from explicit relation sentences.

The resulting Node Model is then available to downstream components (APL generation, DOT export, analysis).

12.2 Canonicalisation and Baseline ACO Files

The front-end provides a canonicalisation function that:

- computes a hierarchical path for each node from the indentation tree (e.g. $[1]$, $[1, 1]$, $[1, 1, 2]$),
- maps each path to a canonical ID using a type-specific prefix:
  - `G` for goals,
  - `S` for strategies,
  - `C` for contexts,
  - `A` for assumptions,
  - `J` for justifications,
  - `E` for evidences,
  - `M` for modules,
- rewrites all node headers to use these canonical IDs, and
- rewrites all relation sentences so that they refer to the canonical IDs.

The output is a *canonical ACO file* that:

- remains valid ACO and is structurally equivalent to the original outline,
- has IDs that are consistent with indentation and unique by construction, and
- is suitable for adoption as a *baseline* version of the outline going forward.

This makes it possible for authors to begin with ad hoc or "flat" IDs and then transition to a stable, canonical numbering scheme once the structure has stabilised.

12.3 Diagnostics and Statistics

In addition to parsing and structural checks, the ACO Frontend computes a set of lightweight diagnostics and summary statistics intended to support iterative authoring and quality review.

These statistics are informational: they do not affect the semantics of the assurance case, but they provide useful signals about structure, completeness, and maturity.

Structural Statistics. The reference front-end also computes simple statistics over the parsed outline, including for example:

- counts of nodes by type (Goals, Strategies, Contexts, Assumptions, Justifications, Evidence, and Modules),
- counts of edges by relation type (e.g. `supported_by`, `in_context_of`),
- the number of top-level nodes, and
- basic tree-shape measures (such as maximum depth).

Evidence Maturity Statistics. In addition to raw Evidence counts, the Frontend may classify the category associated with an Evidence node as: *provisional evidence categories*, reflecting the maturity or status of the evidence artefact as indicated by its label, body text, or annotations.

An evidence category with provisional status may include (but is not limited to):

- **Planned evidence** – evidence that is identified but not yet available,
- **Placeholder category** – a temporary stand-in used during early development before the validation method and evidence-generating procedure has been defined,
- **Candidate evidence** – an artefact that may support the claim but has not yet been fully assessed or accepted, and
- **Accepted evidence** – evidence judged sufficient to discharge the claim.

The precise criteria for these categories are tool- and workflow-dependent and are not fixed by the ACO surface syntax. However, when such categories are recognised, tools should:

- report counts per provisional evidence category in the statistics summary, and
- clearly distinguish provisional evidence from fully accepted evidence.

Use of Statistics. Statistics and diagnostics are intended to:

- help authors identify undeveloped or weakly supported parts of an argument,
- support progress tracking over time, and
- provide quick, high-level insight into the shape and maturity of an assurance case.

Statistics must never be treated as errors, and their absence or suppression (e.g. via command-line options) must not affect correctness.

When parsing or canonicalising an ACO file, the tool can emit:

- error messages for structural problems (as in Section 6),
- warnings about undeveloped goals/modules and other quality issues,
- a compact statistical summary (counts by node type, edge breakdowns, cross-branch relations), and
- per-node hierarchical ID information, including:

  - the canonical hierarchical ID inferred from indentation, and
  - a note if an author-supplied dotted ID does not match the canonical one.

These messages are intended to guide authors during iterative development of patterns and assurance cases.

ASCII Tree View

For quick inspection at the command line, the front-end can generate an ASCII tree view of the parsed outline. A typical rendering might show:

- one line per node,
- indentation using spaces to reflect the structure,
- a single-letter type code (`G`, `S`, `C`, `A`, `J`, `E`, `M`),
- the canonical hierarchical ID, and
- optionally, any author-supplied alias ID in angle brackets (e.g. `<G1>`) when it differs from the canonical ID.

Body text can also be shown, indented one level further than the header line. The ASCII tree view is not part of the ACO language, but it greatly improves usability when working with outlines in a terminal environment.

Tree Verbosity Levels

The ASCII tree view can be rendered at three verbosity levels. Conceptually these form a scale:

Level	Name	Body treatment
0	Skeleton	no body text; only the structural outline (type code, ID, label) is shown.
1	Structure	body snippet only (space permitting); one short line of body text summarising each node.
2	Full	full body text for each node, potentially spanning multiple lines.

Level 2 (*Full*) is the default for interactive use. The CLI may expose these levels via flags such as:

- `-tree-skeleton` (verbosity 0),
- `-tree-structure` (verbosity 1),
- `-tree-full` (verbosity 2, default).

The structural part of each line (tree glyphs, type code, canonical hierarchical ID, optional alias ID, label) is the same at all verbosity levels; only the amount of body text differs.

Command-Line Interface (CLI)

The reference implementation is designed to play the role of a front-end in a toolchain, exposed as a command-line tool that we will refer to here as `aco`. The concrete executable name is not normative, but the following structure and options are.

Subcommands (Informative)

Currently defined subcommands are:

- `aco parse`
  Parse an ACO file, build the Node Model, and report errors, warnings, and statistics. This is the basic "sanity check" command.
- `aco apl`
  Translate an ACO file into the corresponding APL-style block representation.
- `aco aplc`
  Like apl, translate an ACO file into the corresponding APL-style block representation after canonicalizing it.
- `aco canon`
  Canonicalise an ACO file: compute hierarchical IDs from indentation, rewrite all node headers and relation sentences to use the canonical IDs, and emit a canonical ACO file.
- `aco tree`
  Produce an ASCII tree view of the outline, with configurable verbosity for body text.
- `aco dot`
  Export the Node Model as a DOT/Graphviz graph for visualisation. (Not currently implemented.)

A typical synopsis (informative) is:

```
aco parse [OPTIONS] FILE.aco
aco apl   [OPTIONS] FILE.aco
aco aplc  [OPTIONS] FILE.aco
aco canon [OPTIONS] FILE.aco > FILE.canon.aco
aco tree  [OPTIONS] FILE.aco
aco dot   [OPTIONS] FILE.aco > FILE.dot
```

General Options

Common options shared by several subcommands are:

- `-o PATH`, `-output=PATH`
  Write output to `PATH` instead of standard output (where applicable).
- `-quiet`
  Suppress non-essential informational messages (statistics, summaries, etc.) while still reporting errors.
- `-no-stats`
  Disable printing of the statistics summary even when parsing succeeds.

- `-no-hier-ids`
  Disable printing of per-node hierarchical ID diagnostics (where such diagnostics would otherwise be shown).

Not all options are meaningful for all subcommands; the implementation should reject or ignore clearly inapplicable combinations with an explanatory message.

ASCII Tree Verbosity Levels

For the `aco tree` subcommand, the tool supports three verbosity levels for showing node bodies in the ASCII tree view. These correspond to a simple "verbosity scale":

Level	Flag	Body shown in tree
2 (default)	`-full`	full body (all body lines)
1	`-structure`	body snippet only (space permitting)
0	`-skeleton`	no body (headers only)

The default behaviour corresponds to level 2 (`-full`). The other flags select progressively sparser output.

Relation to Other Features

The CLI flags described above are intentionally minimal and independent of the core ACO language:

- They do not change the semantics of ACO or the Node Model, only how information is presented in the ASCII tree or reports.
- They are compatible with canonicalisation: for example, `aco canon` can be used first to normalise IDs, after which `aco tree` (with any verbosity setting) can be applied to the canonical file.
- They are also compatible with the error and warning checks described earlier; verbosity levels control how much body text is shown, not whether problems are detected.

ETB interactive commands (Informative)

The functions exposed through the aco shell-level command are also exposed, though with slightly different syntax, through the ETB's command interpreter both in interactive and scripted modes. The syntax of the ETB versions of the ACO commands is documented in the ETB User Manual.

---

13 Iterator-Bearing Strategies and Validation Boundaries

This section describes an extension that has now been implemented in the ACO processor to further exploit O-ETB APL capabilities.

13.1 Iterator-bearing Strategy syntax

A recent ACO extension is the ability to represent iterator-bearing Strategy nodes corresponding to iterator-bearing strategy forms already meaningful in APL. The syntax is:

```
Strategy [ID] <shortLabel> for each <Var>:<Category> in <Iterand>:
```

For example:

```
Strategy S1 decomp for each N:foundational_plane:node in nodes(S):
  Strat body.
```

The intended interpretation is that:

- the Strategy short label is the part preceding the reserved phrase `for each`,
- the iterator variable is `<Var>`,
- the iterator category or role is `<Category>`, and
- the iterand designator is the term-valued field `<Iterand>`.

This direction is motivated by the need to express in ACO the iterator-bearing strategy forms already present in the APL pattern corpus.

13.2 Validation boundaries

Iterator-bearing syntax exposes an important architectural distinction among three different notions of well-formedness:

1. **ACO-level syntactic well-formedness**: the iterator-bearing header must be definite enough to be parsed and translated deterministically.
2. **APL-level semantic well-formedness**: the resulting iterand term must be recognized as a known and properly formed APL iterator/designator under the current instantiation/provider-layer configuration.
3. **Instantiation-level operational realizability**: the instantiation layer must actually be able to enumerate or compute the values denoted by the iterand against the available models, tools, or derived artefacts.

The ACO processor need not itself execute iterator designators. However, it must preserve enough syntactic structure to translate them into an APL form whose meaning is well-defined for the current O-ETB implementation.

The implementation has been extended to recognize iterator-bearing Strategy headers and translate them into corresponding APL representations while preserving backward compatibility against an automated regression suite.

13.3 Term-valued iterands

The `<iterand>` field may remain semantically opaque to the ACO processor, but must still be syntactically definite enough to be passed through deterministically as a term-like designator. Thus, the processor may validate shape without performing execution. This allows ACO to remain readable while still covering a wider portion of the useful APL space.

13.4 Validation API and provider architecture direction

To support that boundary, a useful architectural direction is for O-ETB to expose an API that allows the ACO layer to ask whether a parsed iterand term is a known and properly formed APL iterator/designator under the current instantiation/provider-layer configuration. Actual execution or enumeration remains a downstream concern. In the longer term, we envision an extensible provider architecture in which the assurance engineer can tailor available iterator primitives to the models, tools, and derived artefacts available in a given project environment.

14 Module declarations, ModuleRef, and GoalRef

An important clarification now settled is the distinction between top-level module declarations and local reference occurrences within an argument subtree.

14.1 Module declarations

A top-level `Module:` unit header introduces a reusable module definition. It is metadata that begins a new unit, analogous in role to a `Case:` header. The body of that unit then contains the argument structure belonging to the module.

14.2 ModuleRef occurrences

By contrast, a local `ModuleRef` occurrence within a Goal/Strategy subtree denotes a pure module or pattern reference leaf. It is not itself the beginning of a new local supporting subtree. Children indented beneath such an occurrence are therefore structurally illegal in the stricter interpretation of the notation.

The preferred local syntax is now:

```
ModuleRef [ID] [<callExpr>]:
```

For example:

```
ModuleRef M0 [n_foundational_plane_node({S},{N})]:
```

The earlier local occurrence form using `Module` is still accepted by the processor for backward compatibility, but new outlines should use `ModuleRef`.

14.3 GoalRef occurrences

A second local reference form is now recognised:

```
GoalRef <TargetId>
```

This is a compact leaf form for local shared-goal reuse. It denotes that the author intends to rely on the same already-established goal subtree elsewhere in the same unit, rather than restating or reinstantiating that subtree.

For example, if `Goal G4` has already been established within the enclosing unit, the author may later write:

```
GoalRef G4
```

The local reference is intentionally author-facing and should not require the author to write the internal ETB term `goal_ref/1`.

14.4 Processing semantics

The corresponding processing behaviour is now part of the reference implementation:

- `ModuleRef` lowers to the existing APL pattern-reference mechanism.
- `GoalRef <TargetId>` lowers to the internal APL form `goal_ref('<TargetId>')`.
- `GoalRef` resolution is currently unit-local and is checked against the local symbol table discipline of the enclosing case or module unit.
- A validity-check hook for shared-goal reuse has been inserted into the downstream execution path. In the current implementation this hook is present but conservative: it succeeds unconditionally, preserving the architectural seam for later refinement.

14.5 Practical consequence for authors and tools

The effect of these additions is to align the ACO surface language more closely with the settled processing model:

- `Module:` defines reusable module units.
- `ModuleRef` invokes those units within a local argument subtree.
- `GoalRef` reuses an already-established local goal subtree within the same unit.

These forms improve clarity for both authors and reviewers by separating definition, invocation, and local structural reuse into distinct author-facing notations.

15 Summary

This ACO v1.2 specification establishes:

- A simple, readable indentation-based outline format for assurance cases.
- A consistent header syntax that merges type, optional ID, and label.
- Optional natural-language relation statements for non-hierarchical links.
- Automated parsing from ACO into an internal Node Model.
- Systematic mappings from the Node Model to:

  - other structured representations of assurance cases and argument patterns, and
  - graphical representations such as DOT/Graphviz.

- Direct generation of APL and graphical representations from a single textual source.

Version 1.1 documented:

- optional case headers for naming and scoping an outline,
- the use of hierarchical identifiers aligned with indentation structure,
- the behaviour of a reference front-end (diagnostics, statistics, undeveloped checks),
- canonicalisation of ACO files to produce structurally consistent, baseline-ready outlines, and
- an ASCII tree view for enhanced visualization of the assurance case.

These features are intended to make ACO more usable in practice without changing the underlying language or the mapping to APL and graphs.

Version 1.2 consolidates these capabilities as baseline practice and formalises the assessment and guidance vocabulary (undeveloped vs. uninstantiated, provisional evidence), along with an explicit view of where state lives across the ACO-to-APL-to-instantiation pipeline.

---

16 Looking Forward

This section presents a *design study* of assessment-related concepts and features associated with ACO. The material in this section describes intended and exploratory capabilities that are not yet implemented in the current ACO Frontend. The development of these and other capabilities will see the modest current Frontend evolve into an advanced ACO Workbench.

The purpose of this discussion is to outline how assessment information, evidence maturity, and argument status could be interpreted and exploited by future tools built on top of the current base. The concepts described here do not affect the core syntax or semantics of ACO v1.2 and should not be assumed to reflect existing Frontend behaviour unless explicitly stated.

16.1 Future Extensions

The ACO specification is intentionally lightweight and focused on a stable and accessible language for authoring assurance case outlines. Nevertheless, a number of extensions are anticipated to support richer workflows and tighter tool integration.

Possible future extensions include, but are not limited to, the following:

- **ACO Workbench.**
  An integrated ACO workbench could combine editing support, live diagnostics, statistics, ASCII tree views, and graphical visualisation into a single interactive environment. This could include IDE plugins, standalone graphical tools, or a web-based front-end built on top of the ACO Workbench capabilities.

- **Direct ACO to DOT rendering.**
  A future version of the ACO Frontend may provide a direct translation from ACO outlines to Graphviz DOT for rapid visualisation during authoring. Such a rendering would be primarily intended for exploratory use and quick inspection of ACO units under development, and would complement (not replace) the DOT rendering of fully instantiated assurance cases provided by O-ETB.

- **Richer evidence annotations.**
  Tool-supported conventions for expressing evidence maturity, review status, confidence levels, or traceability metadata, while preserving the core ACO syntax.

- **Enhanced mapping to downstream representations.**
  More systematic and configurable mappings from ACO to APL and other formal or semi-formal representations, assurance case interchange formats such as SACM[1], and parameterisation and reuse mechanisms.

- **Additional analyses and metrics.**
  Support for more advanced structural or quality analyses over ACO outlines, beyond the lightweight diagnostics defined in the current specification.

These extensions are deliberately deferred to preserve the simplicity and to achieve a stable implementation of ACO v1.2. Their inclusion here signals likely evolution paths rather than commitments to specific features or timelines.

16.2 Assessment Purpose

ACO is not only an authoring language: it is also the entry point for iterative improvement of the assurance case. Authors (and reviewers) require fast feedback about gaps and risks. To that end, the reference front-end should compute assessment signals that are:

- semantically grounded (derived from the Node Model and, when available, instantiated artefacts),
- vector-valued (multiple dimensions, not a single opaque score), and
- usable both in expert mode and for wizard-style author guidance.

16.3 Assessment and Incremental Improvement

16.3.1 Assessment Dimensions (Vector-Valued)

Assessment of an assurance case is multi-dimensional. The following initial dimensions are recommended as a minimal vector for reporting:

1. **Structural completeness**
   - counts and ratio of developed vs. undeveloped goals/modules,
   - fan-in/fan-out anomalies (optional heuristics),
   - presence of disconnected or weakly connected subtrees (where applicable).

[1] Structured Assurance Case Metamodel

2. **Evidence sufficiency**
   - counts of fully developed evidence nodes (when instantiated information is available),
   - counts of provisional-category evidence,
   - counts of goals lacking any evidence-closure path.
3. **Argument sufficiency**
   - presence/absence of strategies where expected (style-dependent),
   - depth and balance of decomposition (heuristic),
   - reuse of repeated substructures (potential pattern candidates).
4. **Assumption exposure**
   - number and placement of assumptions,
   - proportion justified vs. unjustified (based on presence of justification nodes),
   - assumptions supporting undeveloped goals.
5. **Instantiation status**
   - counts of uninstantiated goals/modules (e.g. away goals, unresolved references),
   - counts of instantiated but undeveloped nodes.

Implementations may compute additional dimensions, but these provide an initial stable vocabulary for reports and for future guidance automation.

16.3.2 Confidence as a Derived Measure

ACO does not assign "confidence scores" directly. Instead, confidence is treated as a derived composite measure that must be explainable in terms of the above dimensions. In particular, confidence weakens monotonically with:

- undeveloped goals and modules,
- provisional evidence, and
- excessive reliance on assumptions (especially unjustified assumptions).

This approach aligns with confidence-based assurance case perspectives without introducing pseudo-quantitative numbers detached from semantics.

16.3.3 Evolutionary / Search-Based Improvement Model

The reference workflow treats assurance case development as a state-space exploration:

- **State**: the current case structure plus instantiation status plus assessment vector.
- **Transitions**: transformation operations (e.g. extract modules, replace repeated substructures with pattern references, instantiate provisional evidence categories, decompose undeveloped goals).
- **Evaluation**: compare candidate next steps using assessment deltas.

This model supports both manual expert refactoring and guided "wizard" steps. It does not require AI, but it provides a disciplined interface for incorporating AI proposals in future versions.

16.3.4 Author Guidance Modes

Two guidance modes are anticipated:

- **Expert mode**: full diagnostics; full hint sets; manual selection and backtracking.
- **Wizard mode**: metrics-driven prioritisation ("next best step"); suppression of low-impact hints; explanations of why a step improves assessment.

Wizard decisions must be driven by assessment gradients, not by hard-coded scripts.

16.4 Visualisation Support and Levels

Visualisation exists at two levels in the O-ETB workflow:

1. **ACO-level (authoring/workbench)**
   - ASCII tree view (including forest view) is the primary exploration UI while the case remains in ACO form.
   - Skeleton mode and depth-limiting are recommended for large outlines.
   - Clear markers should be used in reports for: uninstantiated nodes, undeveloped nodes, and provisional evidence.
2. **APL/AC-level (post-instantiation)**
   - Graphviz renderings of instantiated cases (CAPs) are produced by downstream tooling (e.g. `export_ac`) and may be the primary UI in pattern-first workflows.
   - Assessment metrics must remain definable and inspectable on instantiated cases regardless of whether the case originated from ACO.

16.5 Where State Lives in the Toolchain

The persistent "assurance case state" is not a single object owned by the ACO front-end. Instead, it is a progression along the pipeline:

```
aco →[ACO-WORKBENCH] →aco →[ACO-to-APL] →apl →[INSTANTIATE]
→repository(CASES,EVIDENCE) →[EXPORT_AC] →cap(txt, html, svg)
```

This can be modelled abstractly as:

$$\langle \text{ [aco version sequence], final_aco, apl, repo, cap } \rangle$$

At the file-system level, the repository is expected to hold at least:

```
REPOSITORY/
  CASES/      % instantiated ACs (case files, CAPs, exported artefacts)
  EVIDENCE/   % evidence repository (artefacts, status records, provenance)
  OUTLINES/   % ACO versions + workbench state
```

Within `REPOSITORY/OUTLINES`, per-project/per-case trajectories may be maintained as:

```
REPOSITORY/OUTLINES/<Project>/<CaseName>/
  aco_v001.aco
  aco_v002.aco
  ...
  aco_final.aco
  state.pl
```

The `state.pl` file is expected to record workbench state such as: current version pointer, transformation history, cached assessment vectors, and links to instantiated case identifiers under `CASES/`.

16.6 Role of AI Assistance (Out of Scope for current efforts)

AI-assisted transformation suggestions and learned guidance policies are future possibilities but are considered out of scope for v1.2 and the present Workbench implementation pursuits. However, v1.2 defines the assessment vocabulary that would govern any future AI assistance. In particular, AI suggestions must remain advisory and should be evaluated using the same assessment metrics as algorithmic hints.

17 Informative Note on the Emerging ACO Workbench Action Layer

The current ACO processor and notation defined in this specification are being extended in implementation toward a fuller "ACO Workbench" model in which both constructors and transformations are represented explicitly as workbench actions. This section is informative and forward-looking; it does not change the normative core syntax defined above.

17.1 Actions and candidates

A current implementation direction is to represent a concrete next-step operation as:

```
action(Name, Kind, Target, Params)
```

and a recommended next step as:

```
candidate(Action, Score, Rationale, Diagnostics)
```

This provides a common vocabulary for:

- observing what can legally be done next,
- ranking candidate next steps,
- applying a selected operation, and
- reporting both applicability and effect.

17.2 Constructor basis from an initially empty outline

The Workbench direction now explicitly includes primitive constructors that allow an assurance case outline to be grown from an initially empty case shell. The expected progression is that the first applicable constructor is `add_goal`, after which other local constructors become available, including:

```
add_subgoal, add_strategy, add_context, add_justification,
add_assumption, add_evidence, add_module, add_module_ref
```

The primitive constructors are intended to remain simple and local. More elaborate structural improvements remain the role of explicit rewrite transformations such as T1–T7.

17.3 Targets and pairwise loci

The target vocabulary is expected to include both direct node targets and pairwise loci such as `pair/2` for operations that conceptually act over a relation or insertion boundary rather than a single node. Direct goal nesting also remains a valid development step so that a goal may first be decomposed into subgoals and only later clarified by the insertion of an explicit strategy node.

17.4 Current transformation family and diagnostics

The implemented and actively discussed transformation family now includes T1, T2, T6, and T7, with the remaining transformations to be brought into the same action-and-candidate framework. This in turn motivates a richer diagnostic messaging architecture in which the processor can explain:

- why a candidate action is applicable,
- what deficit or opportunity motivated its recommendation,
- what edits were performed, and
- what invariants were revalidated afterwards.

17.5 Expressive completeness and structural abstractions

A near-term representational objective is to bring ACO closer to full practical coverage of the APL constructs currently used in O-ETB. In particular, iterator-like and other structural abstraction mechanisms already meaningful in APL should eventually have disciplined ACO representations. This also motivates renewed alignment among ACO, the ACO-to-APL translation, the translated MILS material, and the pattern corpus maintained under `KB/PATTERNS`.